

MODULE 05

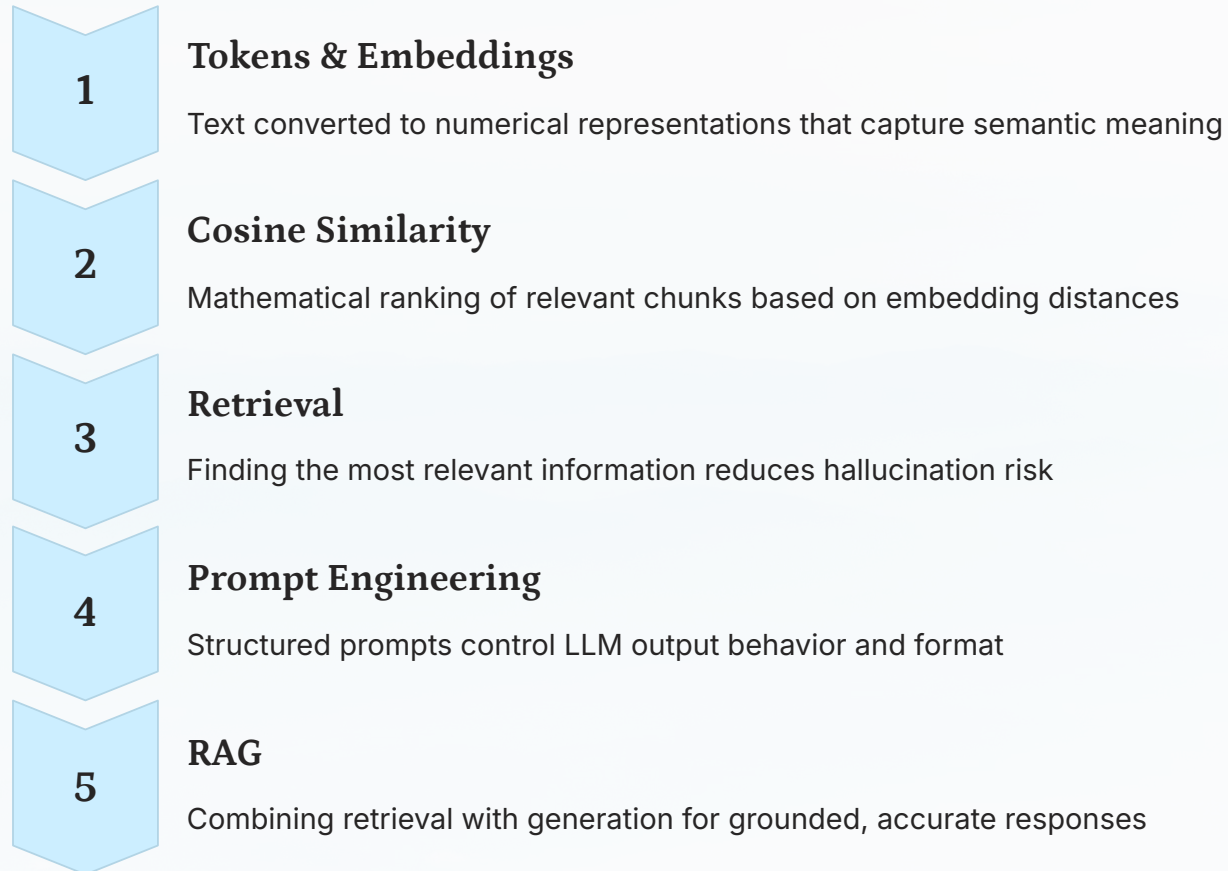
Retrieval-Augmented Generation (RAG) Pipeline

Building an end-to-end RAG system that delivers fact-based answers with citations and built-in safety mechanisms

GenAI, NLP and Prompt Engineering

Quick Recap: Connecting the Dots

Before we dive into RAG pipelines, let's review how the foundational concepts from Modules 2–4 work together to enable retrieval-augmented generation.



Module Overview and Learning Outcomes

This module brings together everything you've learned to build production-ready RAG systems. You'll master the complete pipeline from retrieval to generation, including critical safety mechanisms.

01

Understand RAG Fundamentals

Learn what RAG is, why it's essential for modern AI applications, and how it solves the hallucination problem

02

Implement Retrieval Logic

Retrieve top-k chunks from trusted datasets using similarity scoring and threshold filtering

03

Master Prompt Grounding

Insert retrieved chunks into safe, structured prompt templates that enforce citation requirements

04

Generate Verified Answers

Produce accurate, citation-backed responses that trace every fact back to source documents

05

Handle Edge Cases

Implement robust failure handling for no-answer scenarios, contradictions, and context overflow

What is RAG?

Retrieval-Augmented Generation (RAG) represents a paradigm shift in how large language models deliver information. Instead of relying solely on internal knowledge baked into model weights during training, RAG systems retrieve relevant information from trusted external datasets first, then use that retrieved context to generate responses.

This approach transforms LLMs from knowledge repositories into intelligent reasoning engines that work with your specific, up-to-date information. The result is output that is accurate, verifiable, and grounded in authoritative sources rather than probabilistic guesswork.

Retrieve First

Search trusted private datasets for relevant facts

Ground Context

LLM answers using only retrieved sources

Generate Output

Accurate, verifiable, citation-backed responses

Classical LLM vs RAG: Why RAG is Better

Classical LLM Approach

Traditional large language models generate answers directly from their internal knowledge, which was embedded during pre-training on massive datasets.

While impressive, this approach has critical limitations.

Static Knowledge

Training data becomes outdated quickly—models can't access information published after their training cutoff date

Hallucination Risk

Models may generate plausible-sounding but factually incorrect information when uncertain

No Source Tracing

Impossible to verify where information came from or assess its reliability

RAG-Enhanced Approach

RAG systems retrieve information from verified, up-to-date document collections before generating responses. This retrieval-first approach fundamentally changes the quality and trustworthiness of AI outputs.

Current Information

Access to your latest documents, policies, and data—updated in real-time as your knowledge base grows

Fact-Based Output

Responses grounded in actual retrieved text, dramatically reducing hallucination

Full Traceability

Every claim includes citations linking back to source documents for verification

Core Components of a RAG System

A production RAG pipeline consists of three distinct stages that work together sequentially. Each stage has specific responsibilities and failure modes you must understand to build reliable systems.



Stage 1: Retrieval

Search your document collection to find the most relevant chunks based on semantic similarity to the user's query

- Convert query to embedding vector
- Compare with all chunk embeddings
- Rank by cosine similarity score
- Select top-k most relevant chunks
- Apply threshold filtering to remove weak matches



Stage 2: Grounding

Insert retrieved chunks into a structured prompt template that constrains the LLM to use only provided sources

- Format chunks with source identifiers
- Construct prompt with explicit constraints
- Include citation requirements
- Add refusal instructions for missing info



Stage 3: Generation

Generate a natural language answer that synthesizes information from retrieved sources with proper citations

- LLM processes grounded prompt
- Synthesizes information from chunks
- Adds citation markers to each claim
- Refuses when sources insufficient
- Returns verifiable, traceable output

- ❏ Each stage includes safety mechanisms: retrieval thresholds prevent weak matches, grounding templates prevent fabrication, and generation includes explicit refusal instructions.

Stage 1: Retrieval Deep Dive

The retrieval stage is where semantic search meets practical engineering. Your goal is to find the most relevant chunks from your document collection while filtering out noise that could confuse the LLM or waste precious context window tokens.

This process mirrors the similarity ranking you learned in Module 3, but now we're applying it in a production context with real-world considerations like performance, accuracy, and edge case handling.

- 1

Embed the Query

Convert user's question into a dense embedding vector using the same model that embedded your chunks
- 2

Calculate Similarity

Compute cosine similarity between query embedding and all chunk embeddings in your vector database
- 3

Rank Results

Sort chunks by similarity score from highest to lowest to identify the most semantically relevant matches
- 4

Apply Top-k Filter

Select only the top 3-5 chunks to keep context focused and avoid overwhelming the LLM
- 5

Threshold Filtering

Remove chunks below a minimum similarity threshold (typically 0.60-0.70) to exclude weak matches



Example Retrieval Results

0.89	0.82
Chunk A	Chunk B
Strong match—definitely include	Good match—include
0.76	0.58
Chunk C	Chunk D
Decent match—include	Below threshold—exclude

Stage 2: Grounding – Why It Matters

Grounding is the secret weapon that transforms unreliable LLM outputs into trustworthy, verifiable responses. By inserting retrieved chunks directly into the prompt and adding explicit constraints, you force the model to behave like a careful researcher who only cites what they've actually read.

The Grounding Process

Grounding means taking your retrieved chunks and incorporating them into a structured prompt template that establishes clear boundaries for the LLM. You're not just providing context—you're creating guardrails that prevent the model from inventing information.

- Insert retrieved chunks with clear source identifiers (Source 1, Source 2, etc.)
- Add explicit instructions to use only provided sources
- Require citations for every factual claim
- Instruct the model to refuse answering if sources are insufficient

Why Grounding Works

Without grounding, even when you provide context, LLMs may still blend their parametric knowledge with retrieved information, making it impossible to know what's verified versus fabricated.

Prevents Hallucination

Model can't add information from training data when explicitly constrained to provided sources

Enables Verification

Every fact traces back to a specific retrieved chunk you can inspect

Builds Trust

Users can verify claims by checking cited sources, critical for compliance and sensitive domains

 **Do:** "Answer using ONLY the provided sources. Cite each fact." **Don't:** "Feel free to add relevant information from your knowledge."

Prompt Templates for RAG

A well-designed prompt template is the foundation of a reliable RAG system. Templates provide consistency across thousands of queries, enforce citation requirements, and handle edge cases gracefully. Think of templates as the "operating system" for your RAG pipeline—get this right, and everything else falls into place.

Template Components

Professional RAG templates include several critical sections that work together to constrain model behavior while maintaining natural language quality.

1. **System Role:** Define the assistant's identity and constraints
2. **Retrieved Sources:** Formatted chunks with clear identifiers
3. **Instructions:** Explicit rules about using sources and citations
4. **Refusal Logic:** How to handle insufficient information
5. **User Query:** The actual question being asked

```
You are a helpful assistant. Answer using ONLY the sources below.  
**SOURCES:**  
Source 1: {chunk_1}  
Source 2: {chunk_2}  
Source 3: {chunk_3}  
**RULES:**  
- Use only information from the sources above  
- Cite sources for every claim using (Source N)  
- If sources don't contain the answer, respond: "No answer found in the documents."  
**QUERY:** {user_query}  
**ANSWER:**
```

Why Templates Matter

Consistency

Every query processed identically—no variation in safety mechanisms

Reusability

One template serves chatbots, knowledge bases, and customer support

Maintainability

Update citation format or refusal logic once, applies to all queries

Citation Markers: Showing Your Sources

Citations are the bridge between AI-generated content and human verification. They transform opaque model outputs into transparent, auditable information systems. In regulated industries like healthcare, finance, or legal, citations aren't optional—they're mandatory for compliance and liability protection.

Why Citations Matter



Builds Trust

Users can verify every claim by checking the cited source, critical for high-stakes decisions



Enables Verification

Auditors and compliance teams can trace information flow from source to output



Ethical Transparency

Users deserve to know where information comes from, especially in AI systems

Citation Format Options

Format	Example	Use Case
Parenthetical	(Source 2)	Inline citations in flowing text
Bracketed	[2]	Academic or technical documents
Superscript	claim ²	Minimal visual disruption
Prose	According to Source 2	Natural language contexts

☐ **Example with citation:** "The library opens at 8am on weekdays. (Source 1)"
This format makes verification effortless for users.

No-Answer Detection and Fallback Handling

The most important feature of a production RAG system is knowing when *not* to answer. A system that confidently fabricates information when sources are insufficient is worse than useless—it's dangerous. Robust no-answer detection is what separates toy demos from enterprise-grade systems.

1

Similarity Threshold Check

If all retrieved chunks score below your minimum threshold (typically 0.60-0.70), the retrieval stage returns empty-handed. No chunks means no answer—the system refuses before generation even starts.

2

Hard Prompt Guardrail

Your prompt template includes explicit instructions: "If sources don't contain the answer, respond: 'No answer found in the documents.'" This creates a secondary safety net at the generation stage.

3

Confidence Scoring (Advanced)

Some systems add a third layer: asking the LLM to rate its confidence before responding. Low confidence triggers a refusal even if sources were retrieved.

Safety Mechanism Comparison

Mechanism	When Applied	Example Threshold
Similarity threshold	Retrieval stage	0.60 minimum
Prompt instruction	Generation stage	N/A (text-based)
Empty retrieval check	Between stages	0 chunks returned
Contradiction detection	Post-generation	Advanced systems only

Safe Fallback Response

When the system determines it cannot provide a reliable answer, it must respond gracefully and helpfully:

"I don't have enough information in the available documents to answer this question. Please rephrase your query or contact support for assistance."

This response maintains trust while being honest about system limitations.

Mini RAG Pipeline in Python

Let's see how the three stages come together in actual code. This conceptual demonstration shows how retrieval and prompt engineering "glue" together to create a functioning RAG system. While production systems add error handling, logging, and optimization, the core logic remains this straightforward.

Prompt Construction Logic

The critical step is building the grounded prompt. You're taking the raw retrieved chunks and formatting them into a structured prompt that constrains the LLM effectively.

```
def build_rag_prompt(query, chunks):
    sources_text = ""
    for i, chunk in enumerate(chunks, 1):
        sources_text += f"Source {i}: {chunk}\n\n"

    prompt = f"""You are a helpful assistant.
    Answer using ONLY the sources below.

    **SOURCES:**
    {sources_text}

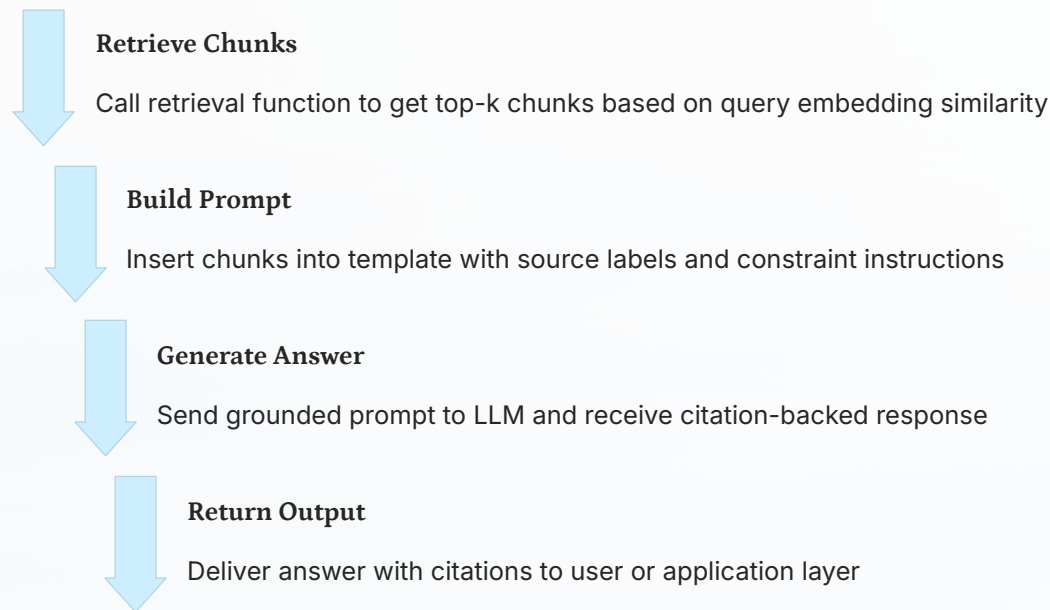
    **RULES:**
    - Cite sources using (Source N)
    - If sources lack info, say:
      "No answer found in documents."

    **QUERY:** {query}

    **ANSWER:**"""""

    return prompt
```

End-to-End RAG Steps



This simple pipeline demonstrates the power of combining retrieval with careful prompt engineering—no complex frameworks required.

Failure Modes in RAG

Understanding how RAG systems fail is just as important as understanding how they succeed. Every production system encounters edge cases, and your job is to anticipate and mitigate these failure modes before they impact users.

Failure Mode	Description	Mitigation Strategy
No Retrieval	No chunks meet similarity threshold, system has nothing to work with	Implement graceful refusal response and suggest query refinement to users
Over-Retrieval	Too many chunks retrieved, including noisy or tangentially related content	Use stricter top-k limits (3-5) and higher similarity thresholds (0.65+)
Contradiction	Retrieved chunks contain conflicting information about the same topic	Deduplication preprocessing, recency weighting, or explicit contradiction detection
Citation Mismatch	LLM generates plausible answer but cites wrong sources or fabricates citations	Post-generation validation to verify claims exist in cited sources
Context Overflow	Retrieved chunks plus prompt exceed model's context window (token limit)	Chunk truncation, dynamic top-k adjustment, or multi-stage summarization

Three Core Mitigation Strategies

Threshold Tuning

Adjust similarity cutoff based on empirical testing—higher thresholds reduce noise but may cause more refusals. Find the balance for your use case.

Top-k Reduction

Start with top-3 chunks instead of top-5. More chunks = more context but also more noise and token usage. Less is often more in RAG.

Deduplication

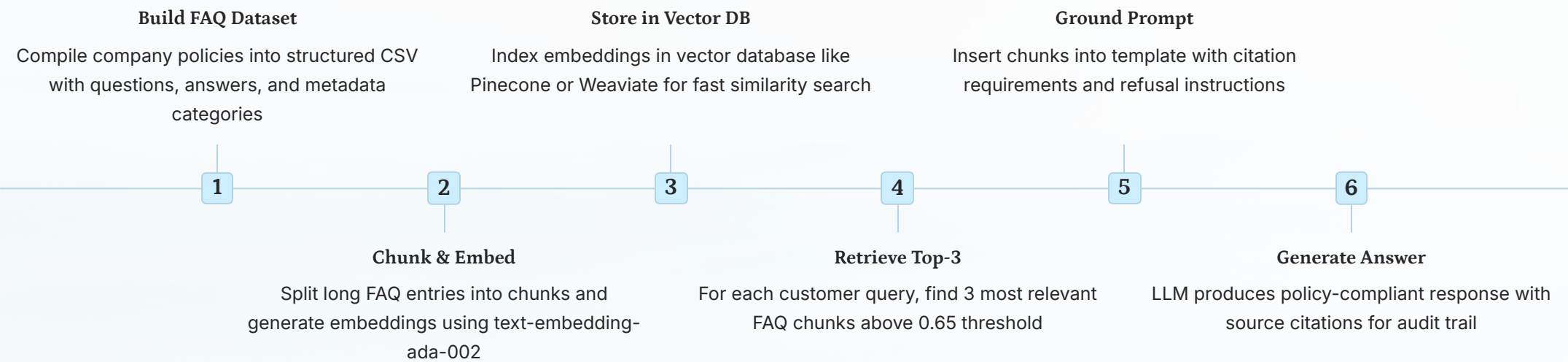
Remove near-duplicate chunks before sending to LLM. Duplicates waste tokens and can confuse the model, leading to repetitive outputs.

Case Study: E-commerce Customer Support RAG Bot

Let's apply everything you've learned to a real-world scenario. This case study demonstrates how RAG solves a common business problem: providing accurate, policy-compliant answers to customer questions 24/7 without human intervention.

Business Challenge

An e-commerce company receives thousands of policy questions daily: return windows, shipping costs, warranty coverage. Human agents are expensive and inconsistent. A classical chatbot hallucinates policies. The solution: a RAG system grounded in verified FAQ documents.



Example Query

"What's your return policy for opened electronics?"

Retrieved Source 1 (0.87): "Electronics can be returned within 30 days if unopened..."

Retrieved Source 2 (0.82): "Opened electronics are eligible for return only if defective..."

Retrieved Source 3 (0.71): "Return shipping is free for defective items..."

Generated Response

Opened electronics can be returned only if they are defective, within our standard 30-day return window. (Source 1, Source 2) If the item is defective, return shipping will be provided at no cost to you. (Source 3) For non-defective opened electronics, please contact customer support to discuss exchange options.

Notice how every factual claim traces to a specific source—making this response auditable and compliant.

Debrief & Homework

Debrief Discussion (10 minutes)

Before you leave, let's solidify the key concepts from this module. These questions will guide our discussion and ensure you're ready to build production RAG systems.

- **Why does RAG reduce hallucination?**

Because grounded prompts constrain the LLM to use only retrieved sources, preventing fabrication from parametric knowledge

- **What roles do top-k and threshold play?**

Top-k limits the number of chunks to prevent noise and token overflow; threshold filters weak matches to ensure relevance quality

- **Why are citations essential?**

Citations enable verification, build trust, support compliance requirements, and make the system transparent and auditable

Homework Assignment (5 minutes)

This hands-on assignment will cement your understanding by building a complete mini RAG pipeline from scratch.

1. **Create a 10-row FAQ dataset** on campus policies or e-commerce topics (CSV format with question/answer columns)
2. **For 3 sample queries, document:**
 - The user query
 - Top 3 retrieved sources with similarity scores
 - Generated answer with citations
 - Verify citations match actual sources
3. **Add a no-answer example** where your query intentionally has no relevant information in your dataset—show how your system refuses gracefully

📄 Submit your FAQ dataset, the 3 complete query examples with retrieval results and answers, plus your no-answer test case. This demonstrates end-to-end RAG pipeline understanding.

Key Takeaways: Building Production RAG Systems

You've now mastered the complete RAG pipeline—from retrieval through generation. These systems represent the current state-of-the-art for building trustworthy, verifiable AI applications that users can rely on for critical decisions.



Retrieval is Foundation

Quality retrieval determines output quality—invest time in proper chunking, embeddings, and threshold tuning



Grounding Prevents Fabrication

Structured prompts with explicit source constraints are your primary defense against hallucination



Citations Enable Trust

Every claim must trace to a source—this isn't optional for enterprise systems handling sensitive information



Refusal is a Feature

Systems that gracefully say "I don't know" when appropriate are more trustworthy than systems that always answer

As you move forward, remember that RAG is not just a technical pattern—it's a paradigm for building responsible AI systems that augment human decision-making without replacing human judgment.



Thank You!

Thank you for completing Module 05! We hope the insights into building production RAG systems were valuable, and we're excited to continue this journey with you.

Next Up: Module 06

Evaluation: Accuracy & Faithfulness of GenAI Systems

Learn how to measure and validate the quality, accuracy, and reliability of your RAG systems in production.